Not Every Notebook Needs Spark: SQL-First Analytics with Pure Python

Jason Romans

Cloud Data and Integration Developer
The Data Shepherd

datashepherd.dev

HI, I'M JASON

THE DATA SHEPHERD



Microsoft® Most Valuable Professional

Honored to be recognized by Microsoft.

I HERD THE DATA

- User Group Organizer
- Nashville, Tennessee
- Fabric + Power BI
- Teaching + Speaking







MY FAVORITE MODEL

TARDIS

TOOLS & ALLIES

Power BI | Microsoft Fabric | DuckDB | Python | SQL | VS Code



datashepherd.dev

DEPARTURES

JAX · 21 OCT 2026

FLIGHT	DESTINATION	GATE	STATUS
PY-01	THE OTHER RUNTIME	A1	SCHEDULED
PY-02	THE DATABASE YOU NEVER INSTALL	A2	SCHEDULED
PY-03	SQL YOU ALREADY KNOW	A3	SCHEDULED
PY-04	EXPLORE BEFORE YOU COMMIT	A4	SCHEDULED
PY-05	THE DECISION TREE	A5	SCHEDULED
PY-06	TAKE IT HOME	A6	SCHEDULED

NOT EVERY NOTEBOOK NEEDS SPARK · SQL-FIRST ANALYTICS WITH PURE PYTHON

NOW BOARDING AT GATE A1

DEPARTURES

JAX · 21 OCT 2026

FLIGHT	DESTINATION	GATE	STATUS
PY-01	THE OTHER RUNTIME	A1	BOARDING NOW
PY-02	THE DATABASE YOU NEVER INSTALL	A2	SCHEDULED
PY-03	SQL YOU ALREADY KNOW	A3	SCHEDULED
PY-04	EXPLORE BEFORE YOU COMMIT	A4	SCHEDULED
PY-05	THE DECISION TREE	A5	SCHEDULED
PY-06	TAKE IT HOME	A6	SCHEDULED

PY-01 · THE OTHER RUNTIME

Sometimes you don’t need Spark.

In Fabric, “notebook” doesn’t mean one thing anymore.

When the question does not need Spark

How many flights departed each airport in January?

Query one month of Parquet data. Inspect the busiest 25 airports.

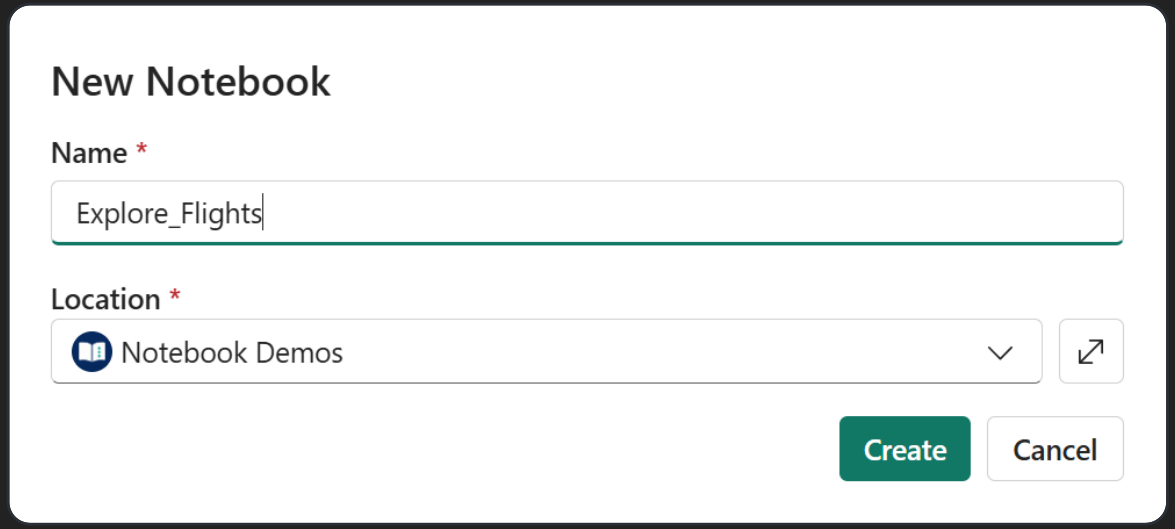
Start with the workload. Choose the compute.

When the workload fits on one node, Pure Python can handle the analysis.

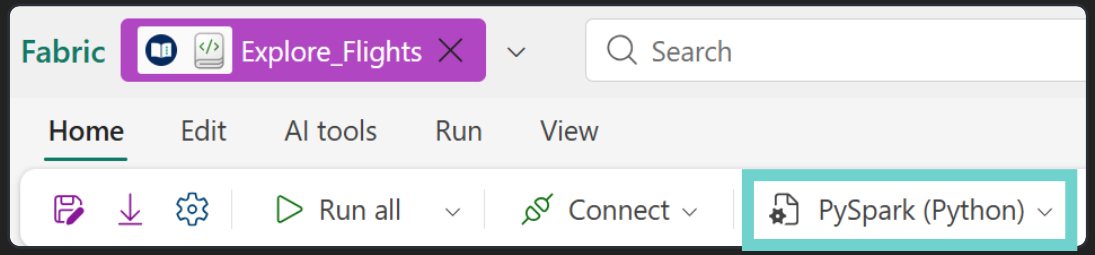
PY-01 · THE OTHER RUNTIME

What happens when you create a notebook?

01 · NAME THE NOTEBOOK



02 · NOTICE THE RUNTIME



Fabric preselects PySpark. You decide whether to keep it.

datashepherd.dev

If Fabric Had an Engine Selection Wizard...

Conceptual mockup — not an actual Fabric screen

Choose the right runtime for your work

1 What are you trying to do?

Select the option that best describes your goal.

- ○ Explore data quickly
- ○ Build a reusable Spark data process
- ◉ Run pandas, Polars, or DuckDB
- ○ Query data with SQL

2 How much compute do you need?

Choose the level of compute that fits your needs.

- ◉ Single machine is probably enough
- ○ Not sure yet
- ○ Distributed compute is required

3 What best fits this workload?

Select the option that most closely matches your scenario.

- ◉ Ad hoc / exploratory
- ○ Repeatable development work
- ○ Scheduled / production execution
- ○ SQL-first analysis

4 Which tools are you planning to use?

Select all that apply.

- ☑ pandas
- ☑ Polars
- ☑ DuckDB
- ☐ PySpark
- ☐ T-SQL

Recommended runtime

Based on your selections, this runtime is the best fit.

Pure Python Notebook — RECOMMENDED

- Fast startup
- Single-node Python runtime
- Great for pandas, Polars, and DuckDB
- No Spark session required

Other options

These may also work, depending on your needs.

Spark Notebook
For distributed Spark work.

SQL Engine / T-SQL
When the work belongs in SQL.

Fabric doesn't ask these questions for you — so you need to ask them yourself.

Jet engine, turboprop, or connecting flight?

The notebook language does not always tell you where the query runs.

JET ENGINE	**Spark notebook**	`Spark cluster`
	PySpark, Scala, Spark SQL, and R share one distributed Spark engine.	
TURBOPROP	**Pure Python notebook**	`Notebook Python process`
	Single-node work. DuckDB is the embedded SQL engine inside the Python process.	
CONNECTING FLIGHT	**T-SQL notebook**	`Warehouse or SQL analytics endpoint`
	A T-SQL notebook runs the query on the selected endpoint. In a Python notebook, %%tsql can call that endpoint too (%%tsql is in preview).	

PY-01 · THE OTHER RUNTIME

WHAT YOU GET

What arrives with a Pure Python notebook

A ready-made environment for exploration, files, DataFrames, and SQL.

EXPLORE DATA

pandas for familiar DataFrame work

Polars for DataFrame-first analysis

QUERY WITH SQL

DuckDB for analytical SQL over local and remote files

READ AND CONNECT

Python libraries for CSV, JSON, Parquet, REST APIs, and files in the Lakehouse

SHARE RESULTS

Tables, DataFrames, charts, and notebook outputs

A ready-to-use Python environment, not a blank runtime to assemble.

The VM is configurable

Default: 2 vCores / 16 GB. The notebook can request a larger single-node VM.

```
%%configure -f
{
    "vCores": 8
}
```

Put this at the start of the notebook.

DEFAULT + CONFIGURATION OPTIONS

2 vCores **16 GB · default**	**4 vCores** **32 GB**	**8 vCores** **64 GB**
16 vCores **128 GB**	**32 vCores** **256 GB**	**64 vCores** **512 GB**

8 GB per vCore. Choose a size supported by your capacity; a changed size can take a few minutes to start.

Microsoft Learn: learn.microsoft.com/fabric/data-engineering/using-python-experience-on-notebook

Two SQL paths in a Python notebook

The key difference: where the query runs.

T-SQL MAGIC

Runs on the Fabric SQL engine

Warehouse, SQL database, or
Lakehouse SQL analytics endpoint

```
%%tsql -artifact lh_pond -type Lakehouse -bind
result
```

DUCKDB

Runs inside the Python process

Reads files and supported
table formats directly

PYTHON CALL

```
result = con.sql("SELECT …").df()
```

Both paths can return a DataFrame to Python.

DEPARTURES

JAX · 21 OCT 2026

FLIGHT	DESTINATION	GATE	STATUS
PY-01	THE OTHER RUNTIME	A1	DEPARTED
PY-02	THE DATABASE YOU NEVER INSTALL	A2	BOARDING NOW
PY-03	SQL YOU ALREADY KNOW	A3	SCHEDULED
PY-04	EXPLORE BEFORE YOU COMMIT	A4	SCHEDULED
PY-05	THE DECISION TREE	A5	SCHEDULED
PY-06	TAKE IT HOME	A6	SCHEDULED

DUCK

Meet DuckDB

An embedded analytics database that runs inside your application or notebook.

RUNS ANYWHERE

Windows, macOS, and Linux

Python, R, Java, Node.js, the command line, and more

QUERIES FILES DIRECTLY

CSV, JSON, and Parquet

Local files or HTTPS. No separate database server to operate.



```
SELECT region, SUM(amount)
FROM 'sales.parquet'
GROUP BY region;
```

In a Fabric Python notebook, Microsoft has already installed it.

Think SQLite — for analytics.

	SQL SERVER	SQLITE	DUCKDB
Runs as	a service	inside your app	**inside your notebook**
Built for	transactions	transactions	**analytics**
Setup	install, configure	none	**none**
You talk to it in	**SQL**	**SQL**	**SQL**

Every one of them answers to SQL. That last row is why you are already qualified to use it.

The data behind the demo

2025 European flight-list data, stored as monthly Parquet files

FLIGHT LISTS

OPDI / Eurocontrol

12 monthly Parquet files
About 16 million flights
Origins, destinations, dates, aircraft

IN YOUR LAKEHOUSE · FILES

```
flights_eu/one_year/
  flight_list_202501.parquet
  ... flight_list_202512.parquet

flights_eu/reference/
  airports.csv
```

TRY IT YOURSELF

0. Setup downloads the data and prepares your Fabric Lakehouse—using a Pure Python notebook, of course.

Why not Polars?

Both are already installed in the Pure Python runtime. **The comparison is about the interface, not a speed test.**

POLARS · FAST, EXCELLENT… AND A NEW API TO LEARN

```
(pl.scan_parquet(f"{F}/flight_list_*.parquet")
   .filter(pl.col("adep").is_not_null() & pl.col("ades").is_not_null())
   .group_by(["adep", "ades"])
   .agg(pl.len().alias("flights"))
   .sort("flights", descending=True).head(10).collect())
```

DUCKDB · THE SQL YOU ALREADY WROTE LAST WEEK

```
SELECT adep, ades, count(*) AS flights
FROM   read_parquet('{F}/flight_list_*.parquet')
WHERE  adep IS NOT NULL AND ades IS NOT NULL
GROUP  BY 1, 2
ORDER  BY flights DESC LIMIT 10
```

Same answer. Same engine class. One of them you can write on day one — that is the whole argument.

PY-02 · THE DATABASE YOU NEVER INSTALL

SQL moves to first class

Polars 2.0 · Released October 6, 2026

Bring the SQL you already know

Polars now treats SQL as a first-class citizen.

Polars brings SQL into your DataFrame workflow.
DuckDB also lets you take that SQL straight to a terminal—no Python required.

Connecting is one call

```
import duckdb

con = duckdb.connect()
```

what you get

an in-memory database

No server and no connection string. DuckDB runs inside the notebook's Python process.

Name the files once

```
FLIGHT_FILES =
"/lakehouse/…/one_year/flight_list_*.parquet"
```

result

a file path pattern

The * matches the monthly Parquet files. Defining the path does not read the data.

Python inserts the path; DuckDB runs SQL

```
con.sql(f"""
    SELECT count(*) AS flights
    FROM read_parquet(
        '{FLIGHT_FILES}',
        union_by_name = true)
""").show()
```

result

one row:
total flights

Python inserts FLIGHT_FILES. DuckDB reads the files; .show() displays the count.

Only the answer comes back

```
departures = con.sql(f"""
    SELECT adep, count(*) AS flights
    FROM read_parquet(
        '{FLIGHT_FILES}',
        union_by_name = true)
    GROUP BY adep
""").df()
```

result

a pandas DataFrame

Use .show() to display SQL results. Use .df() when Python needs the result table. One result row per departure airport.

LIVE DEMO · 1 OF 4

PY-02

Query a Lakehouse file

DuckDB in a Fabric Pure Python notebook

NOTEBOOK · 1. Query a Lakehouse File

01

Real files

A year of flights, live

02

Trust, but verify

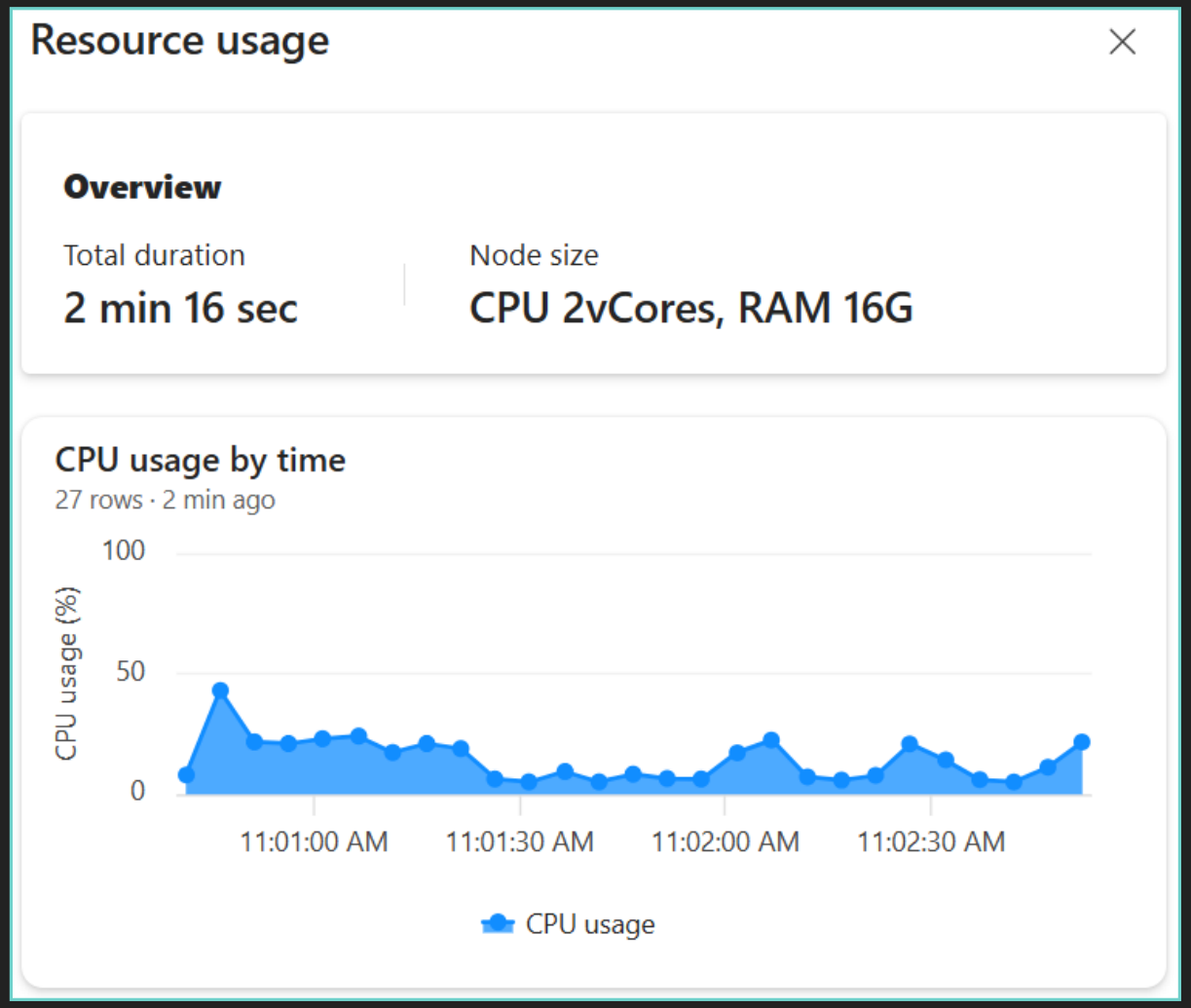
What's in the data

03

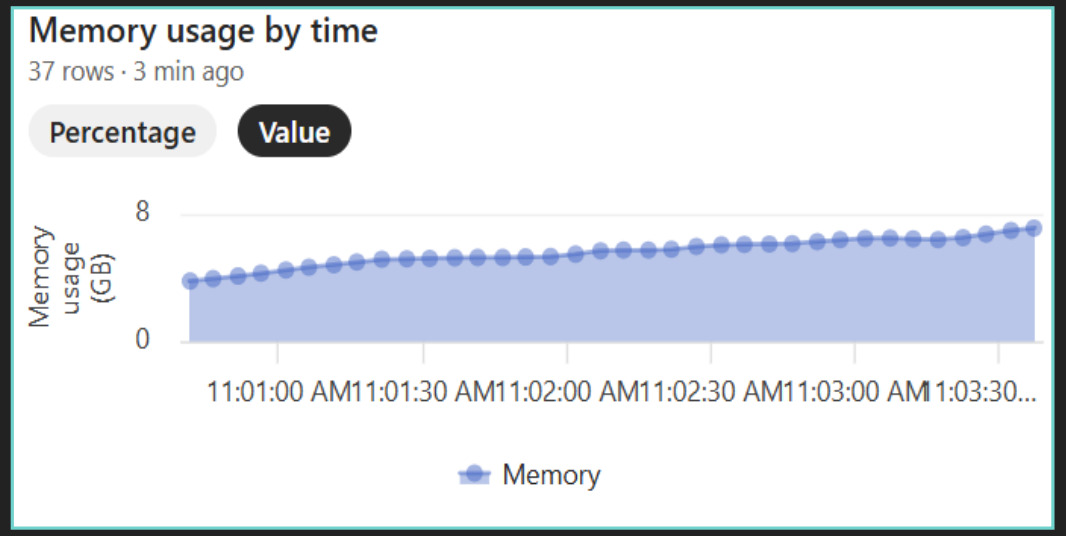
Parquet + CSV

Barcelona in the data

The single node is visible while you work.



2 vCores · 16 GB RAM



CPU and memory over the notebook session

Notebook resource usage, not Fabric capacity cost.

DuckDB is extensible.

THE POINT

Extensions are DuckDB capabilities,
not Python libraries.

The core stays small.
Add formats, data sources, and
domain features when you need them.

HOW YOU GET ONE

```
-- Core extension
INSTALL excel;
LOAD    excel;

-- Community extension
INSTALL zipfs FROM community;
LOAD    zipfs;
```

Core extensions often autoload on first use.
Community extensions use FROM community.

What can DuckDB extensions add?

`httpfs`
query public HTTPS and S3-hosted files

`azure`
access ADLS and Azure Blob Storage

`delta`
read Delta tables without Spark

`iceberg`
read Iceberg tables

`postgres`
query PostgreSQL in place

`excel`
read and write .xlsx

`spatial`
geospatial analysis

`fts`
full-text search

PY-02 · THE DATABASE YOU NEVER INSTALL

Fabric's DuckDB is cleared for upgrades

FABRIC'S DEFAULT

DuckDB 1.4.4 included — this demo uses it as-is.

UPGRADE IF NEEDED

```
%pip install --upgrade duckdb
```

RESTART AND VERIFY

```
notebookutils.session.restartPython()
```

In a new cell after restarting:

```
import duckdb
print(duckdb.__version__)
```

Pin a specific version for repeatable work.

PY-02 · DUCKDB UPGRADES

DuckDB 2.0: Quack and CONNECT

Scheduled October 21, 2026 · tentative release date

An optional server mode

Serve a DuckDB database; CONNECT runs queries remotely.

Faster remote reads

Asynchronous I/O allows more network reads in flight.

Less data to scan

Broader Parquet pruning and partition-aware planning.

BREAKING · GATE CHANGE

PY-ENV

FLIGHT	DESTINATION	GATE	STATUS
PY-02	PYTHON ENVIRONMENTS	A1 → ENV	PRIVATE PREVIEW

Same flight. Better boarding.

In private preview: Fabric Environments for Pure Python notebooks.

UPGRADED DUCKDB · EACH NEW SESSION

```
%pip install --upgrade duckdb
notebookutils.session.restartPython()
```

PINNED VERSION · REUSABLE ENVIRONMENT

Pin library versions
Attach the environment to the notebook

The demo works today. Environments make its dependencies repeatable.

DEPARTURES

JAX · 21 OCT 2026

FLIGHT	DESTINATION	GATE	STATUS
PY-01	THE OTHER RUNTIME	A1	DEPARTED
PY-02	THE DATABASE YOU NEVER INSTALL	A2	DEPARTED
PY-03	SQL YOU ALREADY KNOW	A3	BOARDING NOW
PY-04	EXPLORE BEFORE YOU COMMIT	A4	SCHEDULED
PY-05	THE DECISION TREE	A5	SCHEDULED
PY-06	TAKE IT HOME	A6	SCHEDULED

PY-03 · SQL YOU ALREADY KNOW

SQL You Already Know.
Less Scaffolding Than You Expect.

FAMILIAR SQL

`SELECT, JOIN, WHERE`

`GROUP BY` and aggregates

CTEs and window functions

DUCKDB CONVENIENCES

`GROUP BY ALL`

`EXCLUDE` · `REPLACE` · `RENAME`

`QUALIFY`

`ALIAS REUSE`

`PIVOT`

Standard SQL foundations. A few analytical conveniences. More in the notebook.

GROUP BY ALL

```
SELECT adep, ades,
       count(*) AS flights
FROM read_parquet('{FLIGHT_FILES}',
                  union_by_name = true)
GROUP BY ALL
```

replaces

GROUP BY
adep, ades

Groups by every selected expression that isn't an aggregate.

Reshape SELECT * in place

```
SELECT * EXCLUDE (flt_id)
  REPLACE (try_cast(dof AS DATE) AS dof)
  RENAME (adep AS origin_airport)
FROM read_parquet('{FLIGHT_FILES}',
                  union_by_name = true)
```

result

fewer columns, clearer names

Drop, convert, or rename a few columns and keep the rest of SELECT *.

QUALIFY filters the window

```
SELECT adep, ades,
       row_number() OVER (
           PARTITION BY adep
           ORDER BY count(*) DESC, ades
       ) AS route_rank
FROM read_parquet('{FLIGHT_FILES}',
                  union_by_name = true)
WHERE adep IS NOT NULL
  AND ades IS NOT NULL
GROUP BY adep, ades
QUALIFY route_rank = 1
```

result

busiest route per departure airport

QUALIFY keeps rank 1. The destination code breaks ties between equally busy routes.

Name it once, reuse the name

```
SELECT typecode,
       count(*) AS flights
FROM read_parquet('{FLIGHT_FILES}',
                  union_by_name = true)
GROUP BY ALL
HAVING flights > 100000
```

replaces

repeating
count(*)

Name the calculation once. HAVING reuses the alias.

PIVOT without a column list

```
PIVOT flights_by_month
ON month
USING sum(flights)
GROUP BY adep
```

input: flights_by_month

adep	month	flights
LEBL	2025-01	10
LEBL	2025-02	12
EGLL	2025-01	20

result

adep	2025-01	2025-02
EGLL	20	NULL
LEBL	10	12

DuckDB finds the month columns in the data. Values shown are illustrative.

LIVE DEMO · 2 OF 4

PY-03

SQL with less scaffolding

DuckDB syntax sugar in a Fabric Pure Python notebook

NOTEBOOK · 2. Friendly SQL

01

GROUP BY ALL

Group by what you select

02

QUALIFY

Busiest route from each airport

03

PIVOT

Months become columns

DEPARTURES

JAX · 21 OCT 2026

FLIGHT	DESTINATION	GATE	STATUS
PY-01	THE OTHER RUNTIME	A1	DEPARTED
PY-02	THE DATABASE YOU NEVER INSTALL	A2	DEPARTED
PY-03	SQL YOU ALREADY KNOW	A3	DEPARTED
PY-04	EXPLORE BEFORE YOU COMMIT	A4	BOARDING NOW
PY-05	THE DECISION TREE	A5	SCHEDULED
PY-06	TAKE IT HOME	A6	SCHEDULED

NOT EVERY NOTEBOOK NEEDS SPARK · SQL-FIRST ANALYTICS WITH PURE PYTHON

PY-04 · EXPLORE BEFORE YOU COMMIT

MAIN POINT

Exploration often fits on one node.
The workload determines when to scale.

KEY BEATS

- **Exploratory data analysis (EDA) is a loop:**
  question → query → surprise → better question.
- **Exports often arrive as CSV: infer or supply column types.**
- **Convert once to Parquet.** Query again.

Parquet carries its schema

```
read_csv('{CSV}',
    header = true,
    types = {column_types})

read_parquet('{PARQUET}')
```

difference

Parquet stores its own types

column_types comes from DESCRIBE on the source Parquet. Supply it when reading CSV.

COPY writes the copies

```
COPY (
    SELECT * FROM read_csv(
        '{CSV}', header = true,
        types = {column_types})
) TO '{PARQUET}'
  (FORMAT PARQUET, COMPRESSION ZSTD)
```

result

a Parquet file, Zstandard

COPY … TO writes a query result to CSV, gzip CSV, or Parquet.

Same question, different reader

```
FROM read_csv('{CSV}',
    header = true, types = {column_types})

FROM read_csv('{GZIP}',
    header = true, types = {column_types})

FROM read_parquet('{PARQUET}')
```

result

the same count, three times

Use each reader in the same Barcelona count query. DuckDB reads gzip CSV directly.

LIVE DEMO · 3 OF 4

Keep the next question easy

DuckDB turns exported data into queryable Parquet

NOTEBOOK · 3. CSV to Parquet

01

Three copies

CSV, gzip CSV, Parquet

02

Compare sizes

Same rows, different sizes

03

Same SQL

Barcelona, three ways

Compression saves space. Parquet also preserves types.

WHAT ARRIVES	NEXT QUERY	BETTER USE
`vendor_logs.zip`	Use zipfs or extract	Archive copy
`export.csv.gz`	Decompress and parse when queried	Useful source, repeated cost
`export.parquet`	Query columns directly	Reusable analytical copy

Convert once. Query again.

Compare file sizes and matching results in the notebook. This demo does not benchmark query speed.

DEPARTURES

JAX · 21 OCT 2026

FLIGHT	DESTINATION	GATE	STATUS
PY-01	THE OTHER RUNTIME	A1	DEPARTED
PY-02	THE DATABASE YOU NEVER INSTALL	A2	DEPARTED
PY-03	SQL YOU ALREADY KNOW	A3	DEPARTED
PY-04	EXPLORE BEFORE YOU COMMIT	A4	DEPARTED
PY-05	THE DECISION TREE	A5	BOARDING NOW
PY-06	TAKE IT HOME	A6	SCHEDULED

PY-05 · THE DECISION TREE

ROUTING RULE

Where should this query run?

Follow the execution target, then choose the engine.

01 **Run the query on a Fabric SQL endpoint?** YES — **T-SQL notebook, or %%tsql from Python**

NO ↓

02 **Need distributed compute?** YES — **Spark**

NO ↓

03 **Files and work fit one node** — **Pure Python + DuckDB**

When does one node fit?

For focused SQL exploration, count the time to an answer, including startup.

PURE PYTHON + DUCKDB

01 Focused SQL over Parquet or CSV files

02 One analyst iterating on questions

03 Data and intermediate results fit one node

SPARK

01 Work needs multiple machines

02 Joins or shuffles that need multiple nodes

03 Distributed streaming or existing Spark workloads

04 Scala, R, or existing Spark APIs

SCHEDULING AND ORCHESTRATION STILL APPLY

Pure Python notebooks can still be scheduled and orchestrated in Fabric pipelines.

Choose the engine; retain Fabric scheduling and pipeline orchestration.

DEPARTURES

JAX · 21 OCT 2026

FLIGHT	DESTINATION	GATE	STATUS
PY-01	THE OTHER RUNTIME	A1	DEPARTED
PY-02	THE DATABASE YOU NEVER INSTALL	A2	DEPARTED
PY-03	SQL YOU ALREADY KNOW	A3	DEPARTED
PY-04	EXPLORE BEFORE YOU COMMIT	A4	DEPARTED
PY-05	THE DECISION TREE	A5	DEPARTED
PY-06	TAKE IT HOME	A6	BOARDING NOW

Your data doesn’t have to move first.

One engine. Three ways to reach data.

LAKEHOUSE FILES — Parquet through the notebook mount

```
read_parquet('/lakehouse/default/Files/flights_eu/one_year/flight_list_202501.parquet')
```

LAKEHOUSE TABLE — Delta table, including its transaction log

```
delta_scan('/lakehouse/default/Tables/dbo/flights_202501')
```

PUBLIC AZURE BLOB — Parquet at its existing URL

```
read_parquet('https://YOUR-ACCOUNT.blob.core.windows.net/YOUR-CONTAINER/flight_list_202501.parquet')
```

Fabric is where you are working. It does not have to be where the data lives yet.

One question, many addresses

```
FROM read_parquet('/lakehouse/…')
FROM delta_scan('/lakehouse/…/Tables/…')
FROM read_parquet('https://…')
```

result

same count,
every source

January 2025 Barcelona departures: same SELECT and WHERE, different FROM.

Extensions add readers

```
INSTALL delta;
LOAD delta;

INSTALL httpfs;
LOAD httpfs;
```

unlocks

delta_scan and https:// paths

INSTALL makes an extension available. LOAD turns it on for this connection.

A shortcut gives Blob a Lakehouse path

```
read_parquet(
    '…/Files/blob_flights/…parquet'
)
```

result

same Blob file, Lakehouse path

Create the shortcut, then set RUN_SHORTCUT = True. The file stays in Blob Storage.

LIVE DEMO · 4 OF 4

PY-06

Query data where it lives

DuckDB queries Parquet and Delta where they live

NOTEBOOK · 4. Query Data Where It Lives

01

Lakehouse file

Parquet through the mount

02

Delta table

Through the transaction log

03

Azure Blob

Over HTTPS, then a shortcut

PY-06 · TAKE IT HOME

Choose the right plane for the trip.

Pure Python + DuckDB is a capable route for SQL-led, single-node analytics.

GOOD FIT

Explore files, shape results, and answer a focused analytic question.

WHEN THE TRIP GROWS

Move to distributed compute when the work actually needs it.

NEXT FLIGHT

Take one Parquet file back to Fabric and query it with DuckDB SQL.

github.com/thedatashepherd/purepython-duckdb

Start here.

The session materials and reference docs behind this talk.

START HERE

Session notebooks

Setup notebook and all four demos.

slides.datashepherd.dev/

FLIGHT MANUAL

DuckDB documentation

Friendly SQL, Python, extensions, and file-access guides.

duckdb.org/docs

FABRIC ROUTE

Pure Python notebooks

Microsoft Learn guidance for the Fabric runtime and %%configure.

learn.microsoft.com/fabric/data-engineering/using-python-experience-on-notebook

Keep learning.

Community writing and longer-form DuckDB resources.

LOCAL GUIDE

Miles Cole

Fabric-focused writing, including when DuckDB or Polars fits the work.

milescole.dev/tags

LONGER READING

DuckDB: Up and Running

Wei-Meng Lee O'Reilly book covering fundamentals, files, and Python.

oreilly.com/library/view/duckdb-up-and/9781098159689

BOOKS + TALKS

DuckDB Library

The project's curated collection of books, talks, articles, and podcasts.

duckdb.org/library

Seatback reading.

The Fabric notebook choices behind this session.

CHOOSE THE ENGINE

Fabric notebook selection guide

Compare Pure Python, Spark, and T-SQL by workload.

Open selection guide

SQL NOTEBOOK

T-SQL notebooks

Author T-SQL directly against a Fabric SQL engine.

Open T-SQL guide

PYTHON + T-SQL

%%tsql magic (preview)

Call a Fabric SQL engine from a Python notebook.

Open %%tsql guide

Your first ten minutes.

01

Open

Create a Pure Python notebook and attach a Lakehouse.

02

Query

Run 0. Setup or use your own Parquet file. Ask a question with DuckDB SQL.

03

Simplify

Use one Friendly SQL feature, then save a useful result.

No Spark session to start. One notebook, one file, useful SQL.



THANK YOU

Jason Romans

Email jason@datashepherd.dev
LinkedIn linkedin.com/in/jasonromans
Blog datashepherd.dev

The Data Shepherd
slides.datashepherd.dev

Slides, notebooks & resources
slides.datashepherd.dev